CRS data format (timestreams and sweeps)

This document describes how `citkid.crs` saves CRS data to Zarr: the timestreams written by `CRS.stream` / `CRS.capture_ts`, the sweeps written by `citkid.crs.procedures.target_sweep` / `power_sweep`, the metadata saved with them, and how to load the data back and feed it to `citkid.pipeline`. It is written for lab members taking and analyzing CRS data.

Everything here was checked against the code in `citkid/crs/instrument.py`, `citkid/crs/procedures.py`, `citkid/crs/util.py`, and `citkid/pipeline/sweep_series.py`. Where the code does not pin something down, this document says so. All Zarr usage is the zarr **3.x** API (zarr 3.1.5 in the `citkid` environment), and stores are written in the Zarr v3 format.

Contents:

1. Overview
2. Timestream layout
3. Configuration metadata (`crs_config`)
4. Timestream scaling and units
5. Sweep layout
6. Loading data
7. Feeding a `citkid.pipeline` DataSet
8. How streaming works, performance, and limits
9. Things the format does not record
10. Known inconsistencies

1. Overview

There are three kinds of saved CRS data:

Data	Written by	Main arrays
Timestream	`CRS.write_tones_for_stream` + `CRS.stream`, or `CRS.capture_ts` (both in one call)	`z` (int32, `(2, ntones, nsamples)`), `counts_to_s21`, `dt`, `fres`, `fres_requested`, `ares`
Target sweep	`procedures.target_sweep`	`rough_sweep/`, `gain_sweep/`, `fine_sweep/` subgroups, each with `f` and `z` (`(ntones, npoints)`), and `fres`, `ares`, `qres`, `res_idxs`
Series of target sweeps	`procedures.power_sweep` (or your own loop)	one target sweep per group `set0`, `set1`, ...

`CRS.stream_to_memory` returns a timestream as a NumPy array and saves nothing. `CRS.sweep`, `sweep_span`, `sweep_qres`, and `sweep_full` return arrays and save only the acquisition configuration (to `acq_cfg_grp`, if given); saving the sweep data itself is up to the caller (see 5.3).

Tone index. In every array, axis "tone" is the index into the `fres` array that was passed in (to `write_tones_for_stream`, `capture_ts`, or `target_sweep`). Row `i` of `z`, `counts_to_s21`, `fres`, `fres_requested`, and `ares` all refer to the same tone. The channel map (`ch_map`) records which module and channel each tone was on (see 3.2).

A typical store, as used by `citkid.pipeline.sweep_series` (the group and timestream names, e.g. `ts_00`, are user choices, not fixed by the code):

```
data.zarr/
├── set0/                        # one target sweep (power_sweep naming)
│   ├── (attrs) system configuration, nsamps
│   ├── fres, ares, qres, res_idxs
│   ├── rough_sweep/             # only if npoints_rough is not None
│   │   ├── (attrs) timestamp, fres_update_method, acquisition config
│   │   ├── f, z, fres, chs_module<N>...
```

```
│   ├── gain_sweep/
│   │   ├── (attrs) timestamp, acquisition config
│   │   └── f, z, chs_module<N>...
│   ├── fine_sweep/
│   │   ├── (attrs) timestamp, acquisition config
│   │   └── f, z, chs_module<N>...
│   └── ts_00/                    # a timestream taken with capture_ts
│       ├── (attrs) timestamp
│       ├── z, counts_to_s21, dt
│       ├── fres, fres_requested, ares
│       └── crs_config/
│           ├── (attrs) system + acquisition configuration
│           └── chs_module<N>...
├── set1/
...
```

2. Timestream layout

2.1 Arrays and attributes in the timestream group

`CRS.stream(ts_duration_s, dec_stage, grp, tmp_directory = None, verbose = True)` writes to `grp`:

Name	Kind	dtype	Shape	Contents
`z`	array	int32	`(2, ntones, nsamples)`	Raw I/Q counts. Axis 0 is (I, Q), axis 1 is the tone index, axis 2 is time.
`counts_to_s21`	array	float64	`(ntones,)`	Scale factor from counts to normalized S21, per tone. NaN for tones not on any module.
`dt`	array	float64	`()` (0-d)	Sample time in s, `1 / get_sample_freq(dec_stage)`.
`timestamp`	attribute	str		Local time when `stream` started, `'%Y%m%d,%H:%M:%S'` (e.g. `'20261016,13:00:14'`).
`crs_config/`	group			Acquisition configuration (decimation, sample rate, channel map). See 3.

`write_tones_for_stream(fres, ares, grp = ...)` (and `capture_ts`, which calls it) additionally writes:

Name	Kind	dtype	Shape	Contents
`fres`	array	float64	`(ntones,)`	Tone frequencies actually written, in Hz, after rfmux's frequency adjustment (the docstring says up to about 50 Hz). NaN for tones outside every NCO band (only possible with `allow_missing = True`).
`fres_requested`	array	float64	`(ntones,)`	Tone frequencies passed in, in Hz.
`ares`	array	float64	`(ntones,)`	Tone powers in dBm.
`crs_config/`	group			System configuration (NCOs, firmware, bank, module settings, ...).

Here:

- `ntones = len(fres)`, including tones that ended up on no module.
- `nsamples = int(get_sample_freq(dec_stage) * ts_duration_s)`. The requested duration is not saved; recover it as `nsamples * dt`.
- The sample frequency is `625e6 / (256 * 64 * 2**dec_stage)` Hz (`util.get_sample_freq`):

`dec_stage`	0	1	2	3	4	5	6
sample rate (Hz)	38,147.0	19,073.5	9,536.7	4,768.4	2,384.2	1,192.1	596.0

Tones that are not on any module (missing from `ch_map`) are left as 0 in `z`, and their `counts_to_s21` is NaN, so they come out as NaN after scaling.

Where `fres` lives. `fres`, `fres_requested`, `ares`, and the system half of `crs_config` are written to the group passed to `write_tones_for_stream`, while `z`, `counts_to_s21`, `dt`, `timestamp`, and the acquisition half of `crs_config` are written to the group passed to `stream`. With `capture_ts` these are the same group. If you call `write_tones_for_stream(grp = parent)` once and then `stream(..., grp = parent['ts_0'])`, `stream(..., grp = parent['ts_1'])` (the "several timestreams with the same tones" pattern in `notebooks/crs_daq.ipynb`), the tone arrays and system configuration are only in `parent`, and each `ts_k/crs_config` holds only the acquisition configuration. The `citkid.pipeline.sweep_series` loader reads `ft` from `<ts>/fres` and the stream from `<ts>` (the `capture_ts` layout, default) or from the subgroup `<ts>/<ts_stream>` (see 7). For a series, the subgroup must have the same name in every set.

`stream` refuses to write into a group that already has `z`, `counts_to_s21`, or `dt`; `write_tones_for_stream` refuses a group that already has `fres`, `fres_requested`, `ares`, or `crs_config`. The `timestamp` attribute is not checked and is overwritten if present.

2.2 Chunking and sharding of `z`

`z` is created by `util._create_z_array`:

- `chunks = (1, 1, chunk_N)`: each chunk holds I **or** Q of **one** tone for `chunk_N` time samples.
- `shards = (2, ntones, chunk_N)`: each shard (one file on disk) holds every tone, I and Q, for one block of `chunk_N` samples. Shard `k` covers samples `k * chunk_N` to `(k + 1) * chunk_N`.
- Compression is not set explicitly, so the zarr 3.1.5 default applies: `ZstdCodec(level = 0, checksum = False)` (checked in the `citkid` environment). Compressed size depends on the data.
- Fill value is 0.

Because each chunk is one tone, reading one tone (or a time slice of one tone) only reads that tone's chunks out of each shard, not the whole shard. In zarr 3, `grp['z'].chunks` returns the inner chunk shape `(1, 1, chunk_N)` and `grp['z'].shards` the shard shape.

`chunk_N` comes from `util.get_stream_chunk_length(ntones, nmodules, max_ntones, nframes)`:

1. Start from the larger of `MIN_CHUNK_SAMPLES = 16384` samples (64 KiB per chunk) and the length that makes one shard `MIN_SHARD_BYTES = 128 MiB` (`128 * 1024**2 // (8 * ntones)`).
2. Cap it so that the streaming buffers (`STREAM_NSLOTS = 3` receive blocks per module plus two shards) use at most `STREAM_BUFFER_MAX_FRACTION = 0.25` of the currently available RAM. If this cap pushes `chunk_N` below 16384 (and below `nframes`), a `UserWarning` is raised, since streaming many tones may then drop packets.
3. Cap it to `nframes` (the whole timestream fits in one shard).

Examples, before the memory cap:

`ntones`	`chunk_N` (samples)	shard size
10	1,677,721	128 MiB
100	167,772	128 MiB
1000	16,777	128 MiB
1024 or more	16,384	`16384 * 8 * ntones` bytes (e.g. 512 MiB at 4096 tones)

`chunk_N` is not saved as metadata; read it from `grp['z'].shards[2]` (or `grp['z'].chunks[2]`, which is the same number).

3. Configuration metadata (`crs_config`)

Two functions in `citkid/crs/util.py` save the configuration. They write attributes and refuse to overwrite existing ones.

3.1 System configuration (`util.write_system_cfg_to_zarr`)

Board-wide settings that don't change during a measurement. Saved as attributes:

Attribute	Type	Contents
`nco_module<N>`	float	NCO frequency of module N, in Hz. One per module in `crs.nco_freqs` (this session's modules with an NCO set).
`full_scale_dbm_module<N>`	float	DAC full scale in dBm (read back from the board).
`adc_attenuation_db_module<N>`	float	ADC attenuation in dB.
`nyquist_zone_module<N>`	int	DAC Nyquist zone (1 or 2).
`cable_length_m_module<N>`	float	Cable length the CRS uses for its phase correction, in m.
`adc_calibration_mode_module<N>`	str	`'AUTO'`, `'MODE1'`, or `'MODE2'`.
`adc_autocal_module<N>`	bool	Whether ADC calibration runs continuously.
`firmware_version`	str	CRS firmware version (citkid requires `'1.6.0'`).
`analog_bank_high`	bool	True for modules 5-8, False for 1-4.

Attribute	Type	Contents
`bw`	float	Module bandwidth in Hz (500e6, or 600e6 with extended bandwidth).
`clock_source`	str	`'VCXO'` or `'SMA'`.
`extended_bw`	bool	Extended module bandwidth on/off.
`serial_number`	int	CRS serial number.
`rfmux_version`	str	`'1.4.1'`.
`citkid_version`	str	citkid version at acquisition.

The per-module settings (`*_module<N>`) are the ones in `crs.module_cfg`, set by `CRS.configure_modules` (called by `configure_system` and `attach`).

3.2 Acquisition configuration (`util.write_acq_cfg_to_zarr`)

Settings at the time of a stream or sweep. Saved after the decimation is set:

Name	Kind	Type	Contents
`dec_stage`	attribute	int	Decimation stage (0-6). Always 6 for sweeps.
`sample_freq`	attribute	float	Sample frequency in Hz (`get_sample_freq(dec_stage)`).
`dec_short`	attribute	bool or null	Short packets (128 channels per module) if True, long (1024) if False. `null` in sessions without global control, which can't read it from the board.
`dec_module_idxs`	attribute	list of int or null	Modules the board is streaming. `null` in sessions without global control.
`chs_module<N>`	array	int32, `(n_N,)`	Channel map of module N (below).

Channel map. `chs_module<N>[j]` is the tone index (row of `z`, entry of `fres`) of module N's channel `j + 1`. So module N carries `n_N` tones on channels 1 to `n_N`. A tone index that appears in no `chs_module<N>` was on no module. The arrays are written from `crs.ch_map`, the same dict that `target_sweep` returns and that `capture_ts` / `write_tones_for_stream` accept as `ch_map`. To rebuild it:

```
ch_map = {int(name.removeprefix('chs_module')): cfg[name][:]
          for name in cfg.array_keys() if name.startswith('chs_module')}
```

3.3 Where each half is saved

Data	System configuration	Acquisition configuration
Timestream	attributes of `<tones grp>/crs_config`	`<stream grp>/crs_config` (attributes + `chs_module<N>` arrays)
Target sweep	attributes of the target-sweep group itself (not a `crs_config` subgroup)	each sweep subgroup (`rough_sweep`, `gain_sweep`, `fine_sweep`)

Data	System configuration	Acquisition configuration
`CRS.sweep*` with `acq_cfg_grp`	not saved	`acq_cfg_grp`

Note the asymmetry: timestreams put the configuration in a `crs_config` subgroup, target sweeps put it directly on the group and its sweep subgroups.

4. Timestream scaling and units

`z` holds the raw int32 I/Q values from the streamed packets (the receive code copies `pkt.raw_samples` unchanged). To convert to complex S21:

```
s21 = (z[0] + 1j * z[1]) * counts_to_s21[:, np.newaxis]
```

`counts_to_s21` (`util.get_counts_to_s21`) is, for each tone on a module,

```
counts_to_s21 = VOLTS_PER_ROC / 256 / sqrt(2) / dbm_to_vrms(ares)
dbm_to_vrms(P) = sqrt(50 Ohm * 1e-3 * 10**(P / 10))       # RMS volts
```

with `VOLTS_PER_ROC` and the 50 Ohm termination from `rfmux.core.transferfunctions`, and `ares` the tone power in dBm. The `/ 256` matches the scaling rfmux's own packet parser applies to raw samples. The result is S21 normalized to the tone power: dimensionless, with `20 * np.log10(np.abs(s21))` in dB relative to the tone power (the code calls this dBc). It includes the whole signal chain between the CRS output and input.

Notes:

- rfmux marks `VOLTS_PER_ROC` as empirical, so the absolute level of `|S21|` is approximate; ratios and noise in units of `|S21|` are what you can rely on.
- Phase is the raw demodulated phase, including cable delay (unless a `cable_length_m` correction was set on the board), and is not referenced to anything.
- `CRS.stream_to_memory` returns exactly this scaled array, complex128 of shape `(ntones, nsamples)`, with NaN rows for tones not on any module.
- Sweeps (below) are scaled the same way, except the `/ 256` is not applied because rfmux's `get_samples` already returns scaled values. Both are normalized to dBc, so sweep `z` and timestream `s21` are on the same scale and can be calibrated together. This consistency is what the code intends; it has not been checked against hardware here.

5. Sweep layout

5.1 `target_sweep`

`procedures.target_sweep(crs, fres, ares, qres, res_idxs, grp, ...)` runs up to three sweeps, each with `CRS.sweep_qres`, and saves to `grp`:

On `grp` itself

Name	Kind	dtype	Shape	Contents
system configuration	attributes			See 3.1.
`nsamps`	attribute	int		Samples averaged per sweep point.

Name	Kind	dtype	Shape	Contents
`fres`	array	float64	`(ntones,)`	Tone center frequencies in Hz used for the gain and fine sweeps (after the rough-sweep update, if any). These are the requested centers, not rfmux-adjusted values.
`ares`	array	float64	`(ntones,)`	Tone powers in dBm.
`qres`	array	float64	`(ntones,)`	Span factors: the fine (and rough) sweep span is `fres / qres`.
`res_idxs`	array	int32	`(ntones,)`	Resonator indices. Negative values mark calibration tones, which the rough-sweep update leaves unchanged.

Sweep subgroups `rough_sweep` (only if `npoints_rough` is not None, default None), `gain_sweep` (`npoints_gain`, default 50), `fine_sweep` (`npoints_fine`, default 500). Each has:

Name	Kind	dtype	Shape	Contents
`f`	array	float64	`(ntones, npoints)`	Swept frequencies in Hz, after rfmux's frequency adjustment. Row `i` is tone `i`.
`z`	array	complex128	`(ntones, npoints)`	S21 normalized so that `20 log10|z|` is in dBc (see 4).
`timestamp`	attribute	str		Local start time, `'%Y%m%d,%H:%M:%S'`.
acquisition configuration	attributes + `chs_module<N>` arrays			See 3.2. `dec_stage` is always 6 (~596 Hz).

`rough_sweep` also has `fres` (the tone frequencies before the update) and the `fres_update_method` attribute (`'spacing'`, `'distance'`, or `'minS21'`; see `citkid.res.fres.update_fres`).

Sweep geometry (`CRS.sweep_qres`):

- Each row is `np.linspace(fres + span / 2, fres - span / 2, npoints)`, so **rows are in descending frequency** (downward sweeps). Sort before fitting; the pipeline loaders do.
- Span: `fres / qres` for the rough and fine sweeps, `gain_span_factor * fres / qres` for the gain sweep (default factor 10).

- Each point is the mean of `nsamps` samples at ~596 Hz, i.e. `nsamps / 596` s of averaging per point (plus network overhead).
- Missing tones would be NaN, but `target_sweep` uses the default `allow_missing = False`, so it raises instead of saving NaN rows.

`f` and `z` have `chunks = (1, npoints)` (one tone per chunk) and one shard holding the whole array (`shards = (ntones, npoints)`), with the default Zstd compression.

`target_sweep` refuses to write if `grp` already has `ares`, `qres`, `res_idxs`, `fres`, the `nsamps` attribute, or a sweep subgroup it is about to write.

5.2 `power_sweep` and other series

`procedures.power_sweep(crs, fres, aress, qres, res_idxs, root, ...)` runs one `target_sweep` per power and saves sweep `i` to `root/set{i}` (`citkid.zarr_util.set_name(i)`, e.g. `set0`, `set1`, ...). Each set is in the `target_sweep` format; the powers actually used (after optional shuffling) are in each set's `ares`. Write your own loops (e.g. blackbody sweeps) to `root.require_group(set_name(i))` so the same loaders work.

5.3 Full and other sweeps

`CRS.sweep_full(amplitude, npoints_per_ch, nsamps, ...)` sweeps the whole band of every set NCO with 1024 tones per module and returns **1D** `f, z`, flattened and sorted by frequency ascending (same units as above). No citkid function saves full sweeps, and there is no defined layout for them. `acq_cfg_grp`, if given, only receives the acquisition configuration. Note that `procedures._save_sweep_data` requires 2D arrays, so it can't be used for `sweep_full` output. If you save full sweeps, a natural choice is to create `f` and `z` arrays in the same group you passed as `acq_cfg_grp`, but this is a convention you choose, not one the code defines.

`CRS.sweep`, `sweep_span`, and `sweep_qres` return 2D `(ntones, npoints)` arrays in the same format as the `target_sweep` subgroups, and likewise save only the acquisition configuration.

6. Loading data

```
import numpy as np
import zarr

root = zarr.open_group('data.zarr', mode = 'r')

### Timestream
ts = root['set0/ts_00']
z = ts['z']                                  # zarr.Array, nothing read yet
ntones, nsamples = z.shape[1], z.shape[2]
c2s = ts['counts_to_s21'][:]                 # (ntones,) float64
dt = float(ts['dt'][...])                    # s
fres = ts['fres'][:]                         # written tone frequencies, Hz
t = np.arange(nsamples) * dt                 # sample times from start, s

# One tone, whole timestream (reads only that tone's chunks)
i = 3
zi = z[:, i, :]                              # (2, nsamples) int32
s21_i = (zi[0] + 1j * zi[1]) * c2s[i]        # complex128

# Several tones, first 10 s
idx = np.array([0, 3, 7])
n = int(10 / dt)
zs = z.oindex[:, idx, :n]                    # (2, 3, n) int32
s21 = (zs[0] + 1j * zs[1]) * c2s[idx, np.newaxis]
```

```python
# Whole array, one shard at a time (keeps memory use to one shard)
step = z.shards[2]
for t0 in range(0, nsamples, step):
    block = z[:, :, t0:t0 + step]
    s21_block = (block[0] + 1j * block[1]) * c2s[:, np.newaxis]
    ...  # process s21_block, shape (ntones, <= step)

# Configuration
cfg = ts['crs_config']
sample_freq = cfg.attrs['sample_freq']
nco = {k: v for k, v in cfg.attrs.items() if k.startswith('nco_module')}
ch_map = {int(k.removeprefix('chs_module')): cfg[k][:]
          for k in cfg.array_keys() if k.startswith('chs_module')}
start = ts.attrs['timestamp']              # '%Y%m%d,%H:%M:%S', local time

### Target sweep
ts_grp = root['set0']
fres, ares = ts_grp['fres'][:], ts_grp['ares'][:]
qres, res_idxs = ts_grp['qres'][:], ts_grp['res_idxs'][:]
f_fine, z_fine = ts_grp['fine_sweep/f'][:], ts_grp['fine_sweep/z'][:]
f_gain, z_gain = ts_grp['gain_sweep/f'][:], ts_grp['gain_sweep/z'][:]
order = np.argsort(f_fine[i])               # rows are descending in f
f_i, z_i = f_fine[i, order], z_fine[i, order]
```

Notes:

- Index `z` before converting. `z[...]` or `np.array(z)` on a long timestream loads everything; at 1000 tones, 1e6 samples is 8 GB of int32 and 16 GB as complex128.
- Plain `z[:, idx, :n]` with an integer array also works in zarr 3.1.5 (it is treated as orthogonal indexing); `z.oindex[...]` makes the intent explicit.
- `counts_to_s21` is NaN for tones on no module, so those rows come out NaN.

7. Feeding a `citkid.pipeline` DataSet

`DataSet` does not read CRS arrays itself. Calibration YAMLs start with loading steps that you supply as custom steps; the rest of the calibration uses the parameter names below. For data in the layout above, use `citkid.pipeline.sweep_series.target_sweep_s` (or `target_sweep_source`, which a DataSet embeds so it can be reopened with only its zarr path, or `open_series_datasets` for a whole series).

What the loader reads (`target_sweep_steps(data_path, group, ts = ...)`):

Parameter	Step / mode	Read from	Shape per row
`fres_all`, `qres_all`	`load_global_data` / global	`<group>/fres`, `<group>/qres` (or user arrays)	
`nrows`	`load_global_data` / global	`len(<group>/fres)`	
`dt`	`load_global_data` / global (only if `ts`)	`<stream>/dt`	scalar
`fres`, `qres`, `ares`, `res_idxs`	`load_global_res_data` / global-res	`<group>/fres`, `qres`, `ares`, `res_idxs`	
`ff`, `zf`	`load_sweep_f` / per-row	`<group>/fine_sweep/f`, `z`, row `data_idx`, sorted ascending (optionally averaged by `f_avg`)	`(npoints_fine,)`

Parameter	Step / mode	Read from	Shape per row
`fg`, `zg`	`load_sweep_g` / per-row	`<group>/gain_sweep/f`, `z`, row `data_idx`, sorted ascending	`(npoints_gain,)`
`ft`	`load_ft` / vectorized (only if `ts`)	`<group>/<ts>/fres[data_idx]` (the written tone frequency)	scalar
`zt`	`load_zt` / vectorized (only if `ts`)	`<stream>/z[:, data_idx, :ts_nsamps]`, converted to complex and scaled by `<stream>/counts_to_s21`	`(nsamples,)`

`<stream>` is `<group>/<ts>` (default, the `capture_ts` layout), or `<group>/<ts>/<ts_stream>` if `ts_stream` is given (several streams taken with the same tones, see 2.1).

Which calibration uses what:

Alias (`cal_yaml_path`)	YAML	Needs
`'iq'`	`cal_iq.yaml`	`ff`, `zf`, `fg`, `zg` (no timestream; `ts = None`)
`'ts'`, `'ts_fixed_theta'`	`cal_ts.yaml`	the above plus `ft`, `zt`, `dt`
`'ts_offres'`	`cal_ts_offres.yaml`	the above plus `ft`, `zt`, `dt`

The default calibration steps then use these names: `rmv_gain_f` / `rmv_gain_t` (`xcal.gain.remove_gain` on `ff, zf` and `ft, zt`), `get_xf` (`xcal.calc_x(ff, ft)`), `get_sparper` and `get_sxx` (need `dt`), and so on (`citkid/pipeline/default_steps.py`).

Example, one target sweep with a timestream:

```
from citkid.pipeline.dataset import DataSet
from citkid.pipeline.sweep_series import target_sweep_source

source = target_sweep_source(
    main_dir = '/path/to/data', data_path = 'data.zarr', group = 'set0',
    ts = 'ts_00', ts_nsamps = None,
)
DS = DataSet('/path/to/analysis.zarr', cal_yaml_path = 'ts',
             custom_source = source)
```

Requirements this places on the data:

- The timestream must be taken with the **same tones in the same order** as the sweeps (row `i` everywhere is the same resonator), e.g. by passing the `fres` and `ch_map` returned by `target_sweep` to `capture_ts`.
- `<ts>/fres` must exist, i.e. `ts` must be the group `write_tones_for_stream` wrote to (true for `capture_ts`; see 2.1). If the stream is in a subgroup of it, pass that subgroup's name as `ts_stream`.
- `ft` is the rfmux-adjusted written frequency, which can differ from the requested `fres` and from the fine-sweep grid by tens of Hz.

The generic template `citkid/pipeline/templates/custom_steps_ts.py` shows hand-written loaders of the same kind; its paths are placeholders (see 10).

8. How streaming works, performance, and limits

8.1 Data path

`CRS.stream`:

1. Validates inputs and checks the write rate and free disk space (below).

2. Computes `chunk_N` and creates `z` (in `grp`, or in a temporary store `<tmp_directory>/stream_<YYYYmmdd_HHMMSS_ffffff>.zarr` if `tmp_directory` is given).
3. Sets `timestamp`, sets the decimation (which picks short packets if every module has at most 128 tones, and streams only the modules with tones), and saves the acquisition configuration.
4. Calls `util.capture_to_zarr`: packets are received (with the rfmux packet receiver) into per-module blocks of `chunk_N` samples. When every module has filled block `k`, a writer (`util._zarr_writer`) sorts the channels into tone order using `ch_map` and writes shard `k` of `z`. For a local (directory) store the writer is a separate **process** (spawned, sharing the blocks through shared memory), so compression and disk writes do not slow down receiving; for other stores it is a thread.
5. Stops when every module has exactly `nsamples` samples. Missing packets are **not** filled in (see 9).
6. Copies `z` from the temporary store to `grp` one shard at a time (if `tmp_directory` was given) and deletes the temporary store. It is kept if streaming or copying fails.
7. Writes `counts_to_s21` and `dt`.

If the writer falls `STREAM_NSLOTS = 3` blocks behind, new packets have no free block and are dropped (and counted).

`CRS.stream_to_memory` uses the same receiver but keeps everything in RAM (`util.capture_to_memory`), then converts to complex128 one module at a time.

8.2 Checks before streaming

Check	Function	Limit	Error
Frame rate	`util.check_frame_rate`, in `set_decimation`	`MAX_FRAME_RATE = 3 * 625e6 / (256 * 64 * 2)` ≈ 57,220 frames/s, summed over streamed modules	`ValueError`
Write rate	`util.check_stream_write_rate`, in `stream` and `capture_ts`	`MAX_STREAM_WRITE_RATE = 4096 * 8 * get_sample_freq(4)` = 78.125e6 B/s ≈ 74.5 MiB/s	`ValueError`
Disk space	`util.check_free_space`, in `stream`	`2 * ntones * nsamples * 4` bytes (uncompressed `z`) free on the drive holding `grp` (local stores only) and on `tmp_directory`, if given	`OSError`
Memory (streaming buffers)	`util.get_stream_chunk_length`	buffers ≤ 25% of available RAM	shortens chunks, `UserWarning`
Memory (`stream_to_memory`)	`util.check_memory_for_stream`	estimated peak ≤ 50% of available RAM	`MemoryError`

These limits were measured on 2026-10-16 with `benchmarks/crs_rate_sweep.py` (1 GbE Realtek NIC, AMD Ryzen 7 PRO 5850U, USB SSD at 280 MiB/s); see the comments in `citkid/crs/util.py`. Raise them only after verifying a faster setup with the benchmarks.

Frame rate. Each module sends one packet per sample: one Ethernet frame for a short packet (128 channels), `LONG_PACKET_FRAMES = 6` frames for a long packet (1024 channels). Maximum number of streamed modules:

`dec_stage`	0	1	2	3	4	5	6
short packets (≤128 tones/module)	1	3	4	4	4	4	4
long packets (>128 tones on any module)	0	0	1	2	4	4	4

(4 is the number of modules in an analog bank.) At the limit, the code comments report about 1 lost packet in 360,000 over 5 minutes; at 2 modules on stage 0 the NIC stopped transmitting, which also cut off control of the board.

Write rate. `ntones * 8 bytes * sample rate` must be at most 74.5 MiB/s. `ntones` counts every tone passed in, including tones on no module. Maximum `ntones` per stage: 256 (stage 0), 512 (1), 1024 (2), 2048 (3), 4096 (4) and above; the hardware limit is 1024 tones per module, 4096 per bank.

Data volume. `z` takes 8 bytes per tone per sample before compression:

Example	Uncompressed size
1000 tones, stage 5 (1192 Hz), 10 s	95 MB (9.5 MB/s)
1000 tones on one module, stage 2 (9537 Hz), 60 s	4.6 GB (76 MB/s, just under the write-rate limit)
100 tones on one module, stage 0 (38147 Hz), 60 s	1.8 GB

The free-space check uses the uncompressed size, so it is conservative. When both `grp` and `tmp_directory` are on the same drive, each is checked for the full size separately, not for twice the size.

Tips (from the `CRS.stream` docstring and the limits above): at low decimation stages keep at most 128 tones per module (short packets) and use as few modules as possible; use `tmp_directory` to stream to a fast local drive when `grp` is on a slow or network drive.

9. Things the format does not record

- **Dropped packets.** The number dropped per module is only reported as a `UserWarning` ("The saved samples are not evenly spaced in time"). It is not saved. Because missing samples are not filled in, a timestream with drops has gaps that are invisible in the data and `t = arange(n) * dt` is wrong after the first drop. Keep the warnings (e.g. log them) if this matters.
- **Packet timestamps / sequence numbers.** Not saved. They aren't needed: the CRS streams all modules synchronously, so sample `k` of every module is taken at the same time, and the streaming rate limits (`MAX_FRAME_RATE`, `MAX_STREAM_WRITE_RATE`) keep dropped packets very unlikely.
- **Exact start time.** `timestamp` has 1 s resolution, local time with no time zone, and is taken before the decimation is set and before the first packet, so it precedes the first sample by an unspecified amount.
- **Requested duration and chunk length.** Recover them from `z.shape[2]`, `dt`, and `z.shards[2]`.
- **Full sweeps.** No saving convention (see 5.3).

10. Known inconsistencies

Found while writing this document; not fixed here.

`notebooks/crs_daq.ipynb`

- No open issues: the "Streaming data format" section and the reading example match the code.

Code and docstrings

- `citkid/pipeline/dataset.py:216`: the `custom_source` docstring refers to `citkid.pipeline.loaders.target_sweep_source`; the function is in `citkid.pipeline.sweep_series`, and there is no `loaders` module.
- `citkid/crs/instrument.py:979`: `CRS.sweep` docstring says z is "complex S21 data in dBc"; z is linear S21 normalized so that `20 log10|z|` is in dBc.
- `citkid/crs/instrument.py:1251`: `sweep_full` comment says "other validation is performed in self.sweep_linear"; there is no `sweep_linear` (validation is in `self.sweep`).
- `citkid/crs/util.py:1187-1190`: `_validate_ch_map` docstring says it returns None, but it returns the converted copy of `ch_map`.
- `citkid/crs/util.py:919-971`: `estimate_ts_data_size` reports a "raw streamed data size" (`nmodules * max_ntones` channels, plus a fixed 103 bytes). Since streaming now writes only the `ntones` tones directly to Zarr, only the "processed" size reflects what is saved; the raw figure looks left over from the earlier parser-based approach.
- `citkid/pipeline/templates/custom_steps_ts.py:21-50`: placeholder paths are mixed: `load_global_data` reads `ts_00/dt` and `load_ft` reads `ts_00/fres`, but `load_zt` reads `z` and `counts_to_s21` from the root, and

`load_sweep_f` / `load_sweep_g` read the 2D sweep from the same root `z` (and both from the same `f`). For CRS data, use the `sweep_series.target_sweep_steps` paths in 7.

- Layout asymmetry (design, not a bug): timestreams save the system configuration in a `crs_config` subgroup, while `target_sweep` saves it as attributes of the target-sweep group itself (`citkid/crs/procedures.py:62`).